

LAS Report

INTRODUCTION:

With this report, we want to describe the laboratory experience we have carried out during this course. This laboratory consisted of monitoring a specific DC brushed motor system, connected to both a programmable controlling unit and a microcontroller called LAUNCHXL-F28379D.

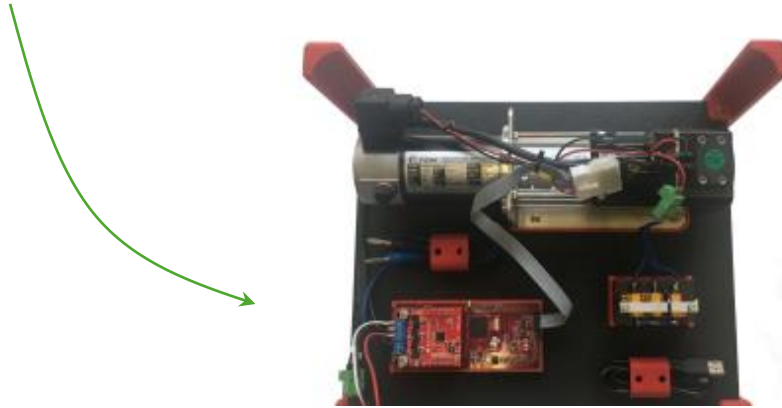


Figura 1. Our setup.

During this course, we have gained confidence with different control techniques for an electric motor, through the tuning of dedicated software regulators on our PC.

We exploited MATLAB & Simulink files to handle discrete time control schemes, and we ran them on the hardware board through the digital communication via cable between the PC and the pins of our “embedded” controlling unit.

Our physical system was composed of a DC brushed motor and a brushless electric motor. Furthermore, the system included also an encoder used to measure the position and the speed of the motor, a drive stage system for the sampling phase and controlling of transistors, a microcontroller with its own programmable control unit and a braking resistive load attachable to our motor.

This “structure” can be also denoted in almost every Simulink file we used:

- We have an area dedicated to the measurement of armature voltage of the motor, armature current of the motor and H bridge’s voltage on the DC bus (“ANALOG READER AREA”).
- Another area represents the incremental encoder included in our motor system, which measures the speed achieved by the motor (“ENCODER AREA”).
- We have also an area in which all the transistors are guided to reach the voltage/current required by PWM unipolar modulation (“PWM AREA”). As we operate only with DC motors, we do not use the third leg of our drive stage. In our case the PWM switching frequency was 10kHz.
- Finally, we have a greater area where anything else useful for our controlling scheme is located, like: reference blocks, enables, proper regulating schemes and so on (“CONTROL SCHEME AREA”).

This is a very basic system, therefore some peculiar crucial points, that usually must be considered as soon as the regulation problem is approached, were not discussed. We chose $T_{sample} = 0.1ms$ as the sampling period, but in a more professional field the sampling time must be calculated in order to meet all the system's requirements. In this way a great compromise between implementation costs and system's performances can be found, leading the digital controller to work very well.

The complete Simulink system, which includes all control areas, is shown in the following *Figure 2*.

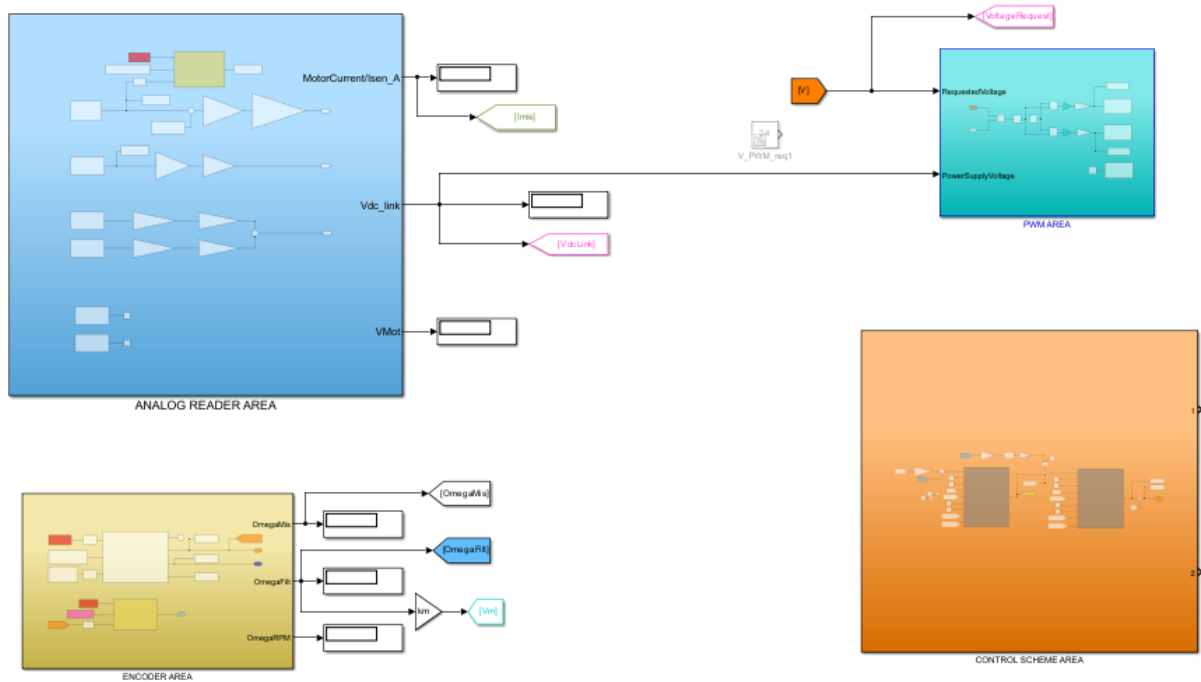


Figure 2. General Simulink scheme.

For this project, we have tested three different control schemes on our DC motor:

- **Proportional-Integral (PI) Cascade control**
- **Deadbeat control**
- **Dahlin control**

We have also estimated the DC motor's parameters using a known technique from the theory, observing improvements in its performances compared to the behavior obtained by using datasheet's parameters.

In particular, the performance of the motor regulated by PI-cascade control was analyzed both with the parameters from datasheet and with the estimated parameters, while the motor's behaviour obtained under Deadbeat and Dahlin controls have been evaluated only by exploiting the estimated parameters and then comparing the real behavior of the motor with the ideal one.

Before describing the control schemes, let's talk about filtering.

FILTERING ACTION:

Filters are essential elements in control systems due to their ability to enhance performance, stability and reliability by managing unwanted signals or noises. One of their primary functions is noise reduction. Control systems often operate in noisy environments where disturbances from electromagnetic interference or electrical noise can corrupt sensor readings or signals. By filtering out high-frequency noise, the system can **focus on relevant signal components**, improving overall accuracy. In our project we have implemented a **First Order Low-Pass Filter** which is located inside the “ENCODER AREA”, as shown in Figure 3.

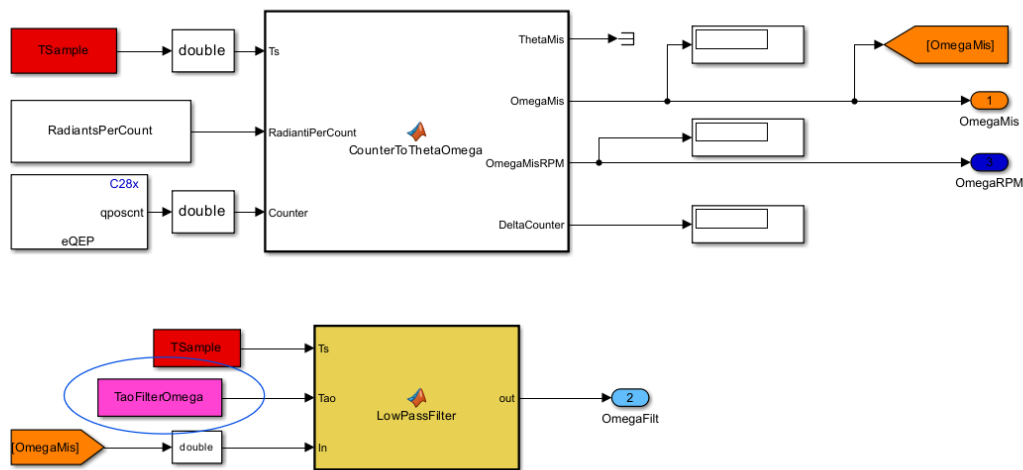


Figure 3. Low-pass filter in the “ENCODER AREA”.

The transfer function of a first order filter system is:

$$G_{filter}(s) = \frac{Y_{filtered}(s)}{X(s)} = \frac{1}{1 + \tau_{filter}s}$$

$Y_{filtered}(s)$ = Output of the filter

$X(s)$ = Input of the filter

Later, we used the **Backward Difference Method** to discretize the system’s filter, obtaining the discretized expression of the output of the filter. Then we transformed this obtained difference

```
function out = LowPassFilter( Ts ,Tao, In)
persistent OutPrecedente
if isempty( OutPrecedente )
    OutPrecedente = 0 ;
end
%Filtro passa basso discretizzato
%con il metodo delle differenze all'indietro

a=Ts/(Ts+Tao);
b=Tao/(Ts+Tao);

out=a*In+OutPrecedente*b;

OutPrecedente = out;
end
```

equation into a MATLAB code (see Figure 4), which can be used whenever we want to insert a low pass filter in our system. The **choice of the time constant** of a filter is important because it directly affects the system's ability to respond to changes in the input signal and it influences how well noises are filtered. If a **low time constant** is selected, the **filter reacts quickly** to changes in the input signal, allowing the control system to follow faster dynamics and respond more rapidly to variations. Nonetheless, a low time constant reduces the filter’s ability to reject high-frequency noise, increasing the system’s susceptibility to disturbances and potentially leading to unstable control actions when noises are present. On the other hand, choosing a **high time constant** reduces the cutoff frequency, which improves the filter’s ability to suppress high-frequency noise,

Figure 4. Low pass filter function.

but in this case the filter will respond more slowly to signal's changes. This slower response can cause delays in the system's reaction to real-time variations, making it less agile. After some tests we chose $\tau_{filter} = 0.001 \text{ s}$ as the time constant of our low pass filter. This implemented filter, with the chosen τ_{filter} , is very important for getting the filtered speed from the encoder, which will be used in almost all the following schemes.

CASCADE CONTROL:

The cascade control is usually based on two (or more) **distinct highly decoupled dynamics**, often indicated as $G_1(s)$ and $G_2(s)$. The DC motor is composed of two dynamics: **electrical (G_{el})** and **mechanical (G_{mecc})** dynamics, as shown in the simulation model in Figure 5.

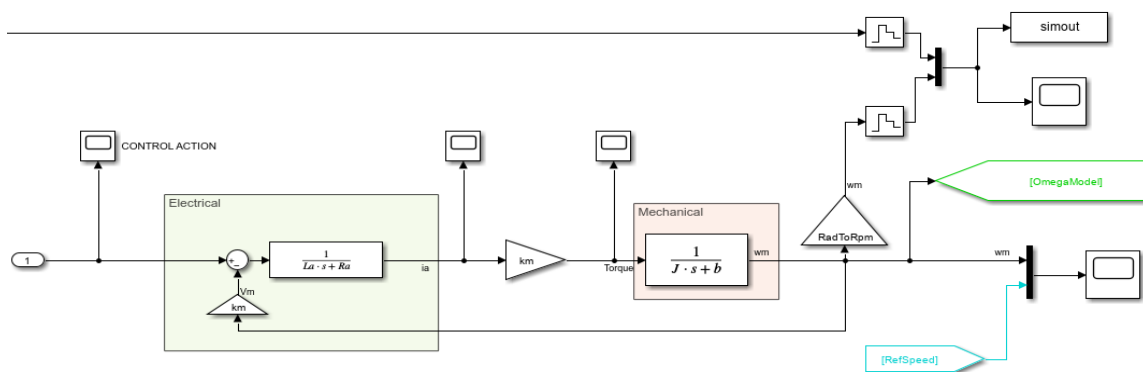


Figure 5. General Simulation Model of the Motor.

The cascade control can be profitably used if one dynamic is faster than the other one and we know that the **electrical dynamics is much faster with respect to the mechanical dynamics**. Thanks to this decoupling, which characterizes the entire system, we have developed the cascade control choosing two **Proportional-Integral (PI) regulators**, which are the **Speed Regulator** and the **Current Regulator** (as depicted in Figure 6), that govern the mechanical and electrical dynamics respectively.

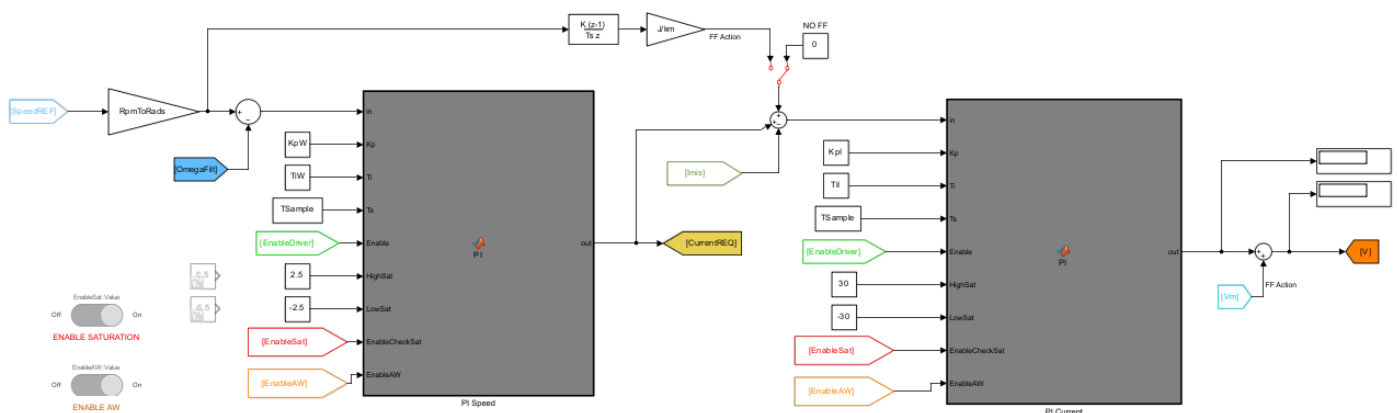


Figure 6. PI Cascade Control.

The general transfer function for a PI-regulator is:

$$R(s) = K_p \left(1 + \frac{1}{T_i s} \right) = \frac{K_p}{T_i} \left(\frac{1 + T_i s}{s} \right)$$

Subsequently we discretized $R(s)$, using the **Tustin's Method**, and then we obtained the discretized expression of the output of a PI controller, which was again converted into a MATLAB function (as shown in *Figure 7*) in order to use it every time we wanted to insert a PI regulator in the system (in our case the speed regulator and the current regulator).

```
function out = PI(in, Kp, Ti, Ts, Enable)
persistent in_prec out_prec

if isempty(out_prec)
    out_prec=0;
end

if isempty(in_prec)
    in_prec=0;
end

% G(s)=(Kp/Ti)*((1+Ti*s)/s) (Tustin)
% We run control only if motor is running
if(Enable==1)
    %I Calculate the output
    out = Kp*Ts/(2*Ti)*(in+in_prec) + Kp*(in-in_prec) + out_prec;

else
    out=0;
    in=0;
end
out_prec = out;
in_prec= in;
end
```

Figure 7. PI Digital implementation.

CURRENT REGULATOR:

Recalling the transfer function of a PI regulator:

$$R_i(s) = K_{pi} \left(1 + \frac{1}{T_{ii} s} \right) = \frac{K_{pi}}{T_{ii}} \left(\frac{1 + T_{ii} s}{s} \right)$$

we have tuned the current controller, obtaining the two essential corresponding control parameters, which are the proportional gain K_{pi} and the integral time T_{ii} .

The electric plant dynamics is described by the following transfer function:

$$G_{el}(s) = \frac{I_a(s)}{V_a(s)} = \frac{1}{R_a + sL_a} = \frac{1/R_a}{1 + \tau_{el}s}; \quad \tau_{el} = \frac{L_a}{R_a}$$

The **current open loop transfer function** is:

$$G_i(s) = R_i(s)G_{el}(s) = \frac{K_{pii}}{T_{ii}} \left(\frac{1 + T_{ii} s}{s} \right) \frac{1/R_a}{1 + \tau_{el}s}$$

Then, setting T_{ii} **equal to** τ_{el} , we will have a pole-zero cancellation in the open loop transfer function which becomes:

$$G_i(s) = R_i(s)G_{el}(s) = \frac{K_{pi}}{T_{ii}R_a s}$$

The **current closed loop transfer function** is:

$$F_i(s) = \frac{G_i(s)}{1 + G_i(s)} = \frac{K_{pi}}{T_{ii}R_a s} \frac{T_{ii}R_a s}{T_{ii}R_a s + K_{pi}} = \frac{1}{1 + \frac{T_{ii}R_a}{K_{pi}} s} = \frac{1}{1 + \tau_i s}$$

$$\text{where } \frac{T_{ii}R_a}{K_{pi}} = \tau_i$$

By assigning a desired time constant τ_i for the current loop, we obtained the value of K_{pi} :

$$K_{pi} = \frac{T_{ii}R_a}{\tau_i} \text{ and recalling that } T_{ii} = \tau_{el} = \frac{L_a}{R_a} \rightarrow K_{pi} = \frac{L_a}{R_a} \frac{R_a}{\tau_i} = \frac{L_a}{\tau_i}$$

The current PI parameters are defined as $T_{ii} = \tau_{el} = \frac{L_a}{R_a}$, $K_{pi} = \frac{L_a}{\tau_i}$

Initially, for the PI-Cascade control we have considered the datasheet's parameters of the motor, which are the following:

$$\begin{cases} L_a = 4.1 \cdot 10^{-3} \text{ H} \\ R_a = 3.2 \Omega \end{cases} \rightarrow \tau_{el} = 1.28 \text{ ms}$$

$$\tau_{el} = \frac{L_a}{R_a}$$

Finally, setting $\tau_i = 1 \text{ ms}$, numeric values of CURRENT PI parameters are found:

- $T_{ii} = \frac{L_a}{R_a} \approx 0.0013 \text{ s}$
- $K_{pi} = \frac{L_a}{\tau_i} \approx 4.1$

SPEED REGULATOR:

Recalling the transfer function of a PI regulator:

$$R_\omega(s) = K_{p\omega} \left(1 + \frac{1}{T_{i\omega} s} \right) = \frac{K_{p\omega}}{T_{i\omega}} \left(\frac{1 + T_{i\omega} s}{s} \right)$$

we have tuned the speed controller, obtaining the two essential corresponding control parameters, which are the proportional gain $K_{p\omega}$ and the integral time $T_{i\omega}$.

The mechanical plant dynamics is described by the following transfer function:

$$G_{mecc}(s) = \frac{\Omega(s)}{C_m(s)} = \frac{1}{b + sJ} = \frac{1/b}{1 + \tau_{mecc} s}; \tau_{mecc} = \frac{J}{b}$$

$J = \text{system's inertia}; b = \text{viscous coefficient of friction}$

Subsequently we made a reasoning: if we assume that the dynamics of the closed loop electric system is much faster than the mechanical one, then both regulators work on **highly decoupled temporal dynamics**, so we can neglect the presence of the the term $F_i(s) = \frac{1}{1+\tau_i s}$ (which represents the closed loop transfer function of the current loop) in the calculation of the open loop transfer function of the speed loop.

Therefore, considering also $k_m = 0.12$ (“motor constant” = constant of proportionality between the armature current and the motor torque, from datasheet), we obtained the following **speed open loop transfer function**:

$$G_\omega(s) = R_\omega(s)F_i(s)k_m G_{mecc}(s) \approx R_\omega(s)k_m G_{mecc}(s) = \frac{K_{p\omega}}{T_{i\omega}} \left(\frac{1 + T_{i\omega}s}{s} \right) k_m \frac{1/b}{1 + \tau_{mecc}s}$$

Then, setting **$T_{i\omega}$ equal to τ_{mecc}** , we will have again a pole-zero cancellation in the open loop transfer function which becomes:

$$G_\omega(s) = \frac{K_{p\omega}k_m}{T_{i\omega}bs}$$

The **speed closed loop transfer function** is:

$$F_\omega(s) = \frac{G_\omega(s)}{1 + G_\omega(s)} = \frac{1}{1 + \frac{T_{i\omega}b}{K_{p\omega}k_m}s} = \frac{1}{1 + \tau_\omega s}$$

$$\text{where } \frac{T_{i\omega}b}{K_{p\omega}k_m} = \tau_\omega$$

By assigning a desired time constant τ_ω for the speed loop, we obtained the value of **$K_{p\omega}$** :

$$K_{p\omega} = \frac{T_{i\omega}b}{\tau_\omega k_m} \text{ and recalling that } T_{i\omega} = \tau_{mecc} = \frac{J}{b} \rightarrow K_{p\omega} = \frac{J}{b} \frac{b}{\tau_\omega k_m} = \frac{J}{\tau_\omega k_m}$$

The speed PI parameters are defined as: **$T_{i\omega} = \tau_{mecc} = \frac{J}{b}$, $K_{p\omega} = \frac{J}{\tau_\omega k_m}$**

Then, we considered again the correspondents datasheet parameters, which are:

$$\begin{cases} J = 7.32 \cdot 10^{-5} \text{ kg} \cdot \text{m}^2 \\ \tau_{mecc} = 85 \text{ ms} \end{cases} \rightarrow b = 8.61 \cdot 10^{-4} \frac{\text{Nm}}{\text{rad/s}}$$

$$b = \frac{J}{\tau_{mecc}}$$

Finally, we set **$\tau_\omega = 10 \text{ ms}$** (in fact, we know that the electrical dynamics is faster with respect to the mechanical dynamics, so we must have $\tau_i \ll \tau_\omega$), obtaining the numeric values of the parameters belonging to the SPEED PI controller, which are:

- $T_{i\omega} = \frac{J}{b} \approx 0.085 \text{ s}$
- $K_{p\omega} = \frac{J}{\tau_\omega k_m} \approx 0.061$

After all these computations, we were ready to check the system's step response, **providing a step input reference equal to 1000 rpm**, as shown in *Figure 8*.

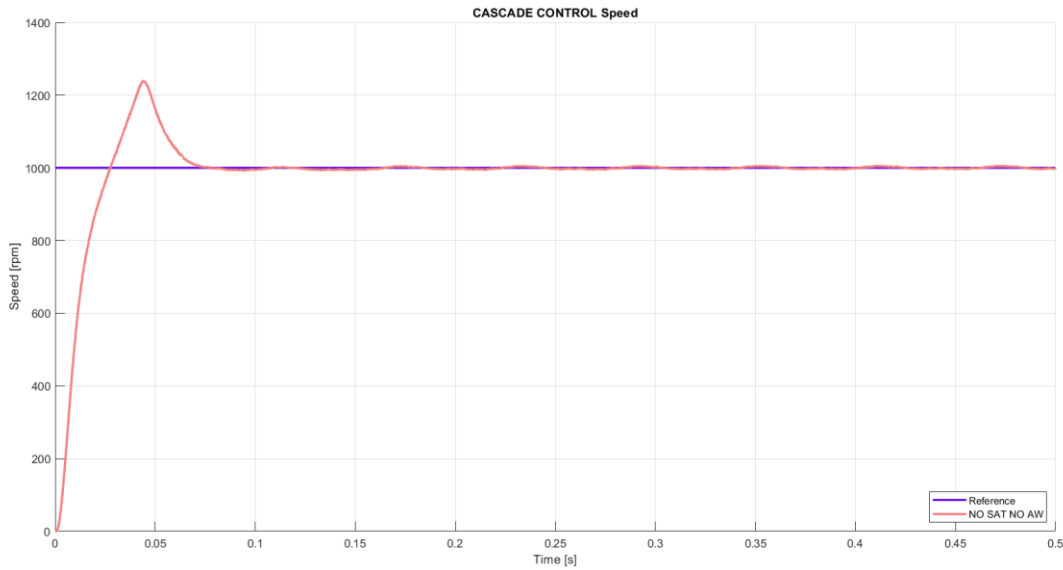


Figure 8. Speed response, NO SAT, NO AW with PI-Cascade control.

As we can see from the red curve, the speed response of our real motor presents an important overshoot that must be cancelled in order to obtain better tracking performances. The correspondent current plot is shown here:

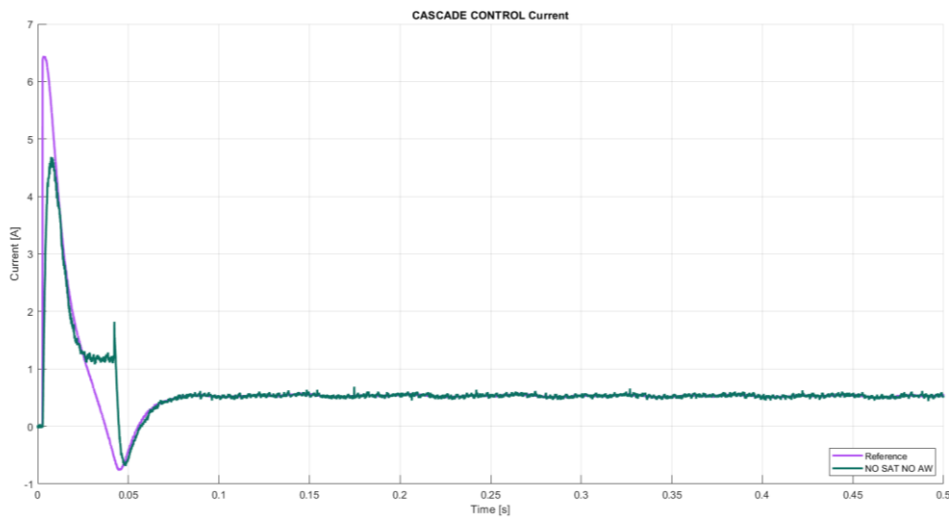


Figure 9. Current response, NO SAT, NO AW, with PI-Cascade control.

From the current plot we can observe that the reference trajectory is not followed in a perfect way, especially in the starting phase. Thus, we have obtained two undesired behaviours in both speed and current plots, which can be related to voltage or current saturation limits. We know that most of the electric motors have some **physical limits** in terms of voltage and current absorption and they can be dangerous if they are not carefully considered. For our DC motor, the “OverCurrentProtection”, also known as “OCP value”, was **2.5A** and the voltage saturation limit was

fixed at **30V**. Having this current and voltage limits means often getting much **slower settling time of the signal**, even if overshoot's issue may be solved, as we can see from the *Figure 10*.

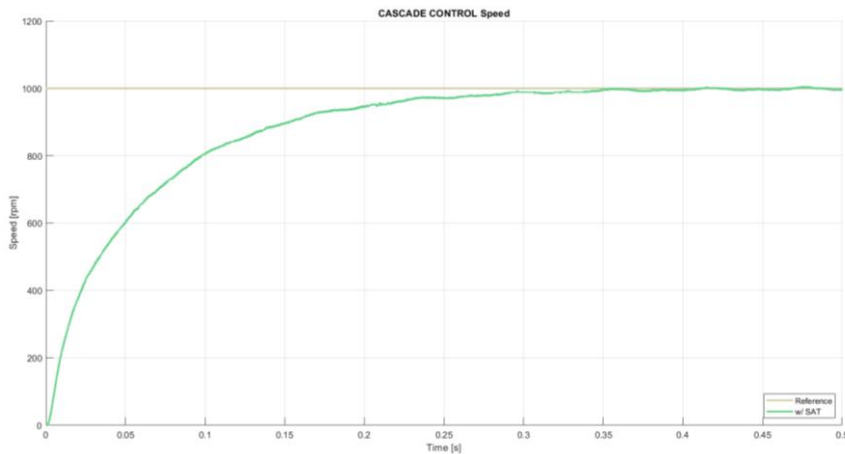


Figure 10. Speed response + saturation with PI-Cascade control.

The reason behind this problem is that PI controllers have, for definition, an **integral action** which can cause the so called “**windup phenomenon**”. Basically, the integrator does not stop integrating the tracking error even if the input is already saturated. This leads to an integrator’s overcharging and therefore, since the integrator will take some time to discharge, more time will be needed by the system to reach the reference, and this can bring into game **instability problems**. In order to avoid the windup effect, we introduced one of the antiwindup schemes seen from the theory: this implementation permits to stop the integration of the error carried out by the regulator as soon as the control action saturates.

```
function out = PI(in, Kp, Ti, Ts, Enable, HighSat, LowSat, EnableCheckSat, EnableAW)
persistent in_prec out_prec

if isempty(out_prec)
    out_prec=0;
end

if isempty(in_prec)
    in_prec=0;
end

% G(s)=(Kp/Ti)*((1+Ti*s)/s) (Tustin)
% We run control only if motor is running
if(Enable==1)
    %I Calculate the output
    out = Kp*Ts/(2*Ti)*(in+in_prec) + Kp*(in-in_prec) + out_prec;
    if(EnableCheckSat == 1)

        %check for saturation
        if out > HighSat
            out = HighSat;
            if (EnableAW==1)
                % I calculate the input that gives me the saturated output
                in=(out-Kp*Ts/(2*Ti)*in_prec + Kp*in_prec - out_prec)/(Kp*Ts/(2*Ti)+Kp);
            end
        elseif (out < LowSat)
            out = LowSat;
            if (EnableAW == 1)
                in=(out-Kp*Ts/(2*Ti)*in_prec + Kp*in_prec - out_prec)/(Kp*Ts/(2*Ti)+Kp);
            end
        end
    end
else
    out=0;
    in=0;
end
out_prec = out;
in_prec= in;
end
```

Figure 11. PI implementation with AntiWindup code on Simulink.

By **introducing this antiwindup control scheme** and **maintaining the current saturation limit at $\pm 2.5A$** as before, we finally obtained the plots in *Figure 12* and *Figure 13*, in terms of speed and current responses respectively.

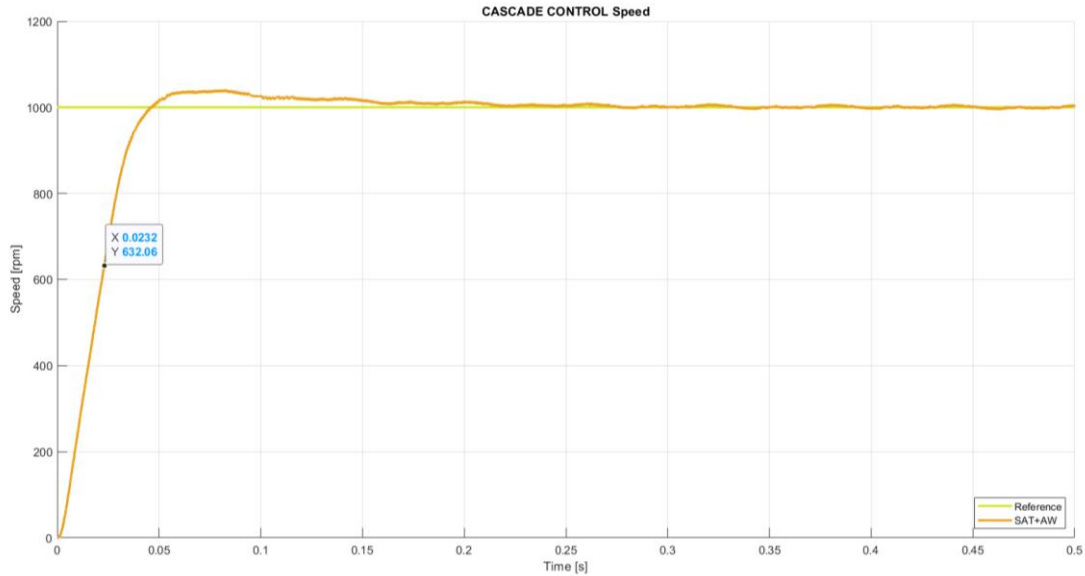


Figure 12. Speed response + saturation + antiwindup with PI-Cascade control.

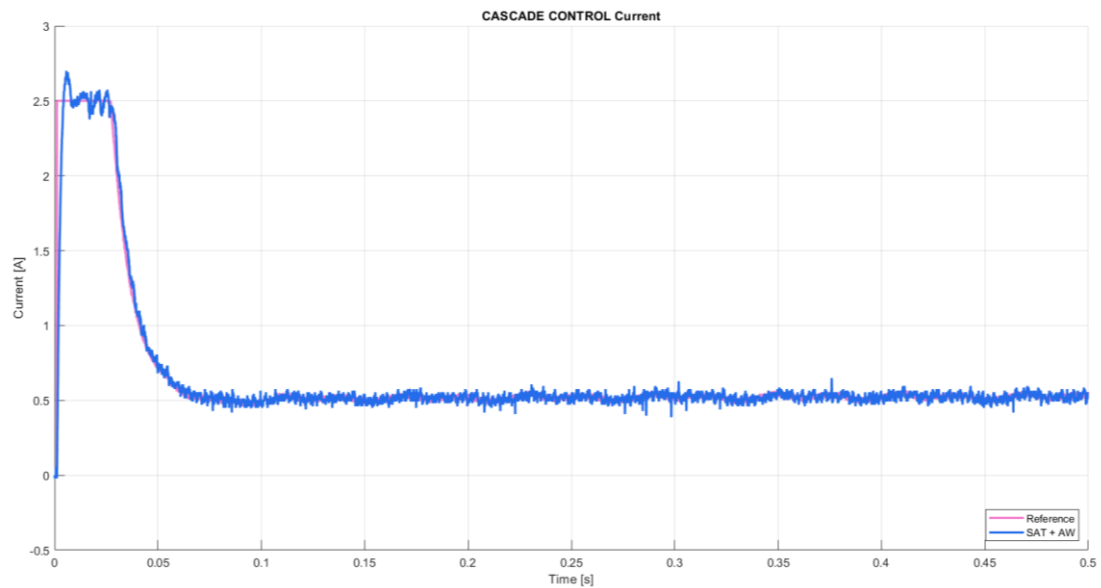


Figure 13. Current response + saturation + antiwindup with PI-Cascade control.

Now we can observe that the speed response is much better in terms of rising and settling time with respect to the previous one with only the saturation limit and this is thanks to the introduction of the anti-windup. Moreover, as we can see from *Figure 13*, now the current follows better the reference. However, speed dynamic performances are not satisfied at all, since the response's time constant is almost **twice** the one we wanted. In fact, as we can see from *Figure 12*, when we are at the 63% of the steady state value (630 rpm) we read $\tau_{\omega} \approx 23 \text{ ms}$ and not $\tau_{\omega} \approx 10 \text{ ms}$, as required.

A simple attempt to achieve speed dynamic requirements is to make our motor able to absorb more current, so that more power will guide the motor and consequently the reference speed (1000 rpm) will be reached in a faster way. We moved the OCP saturation limit up to **6.5A** and we compared the results, as shown in *Figure 14*.

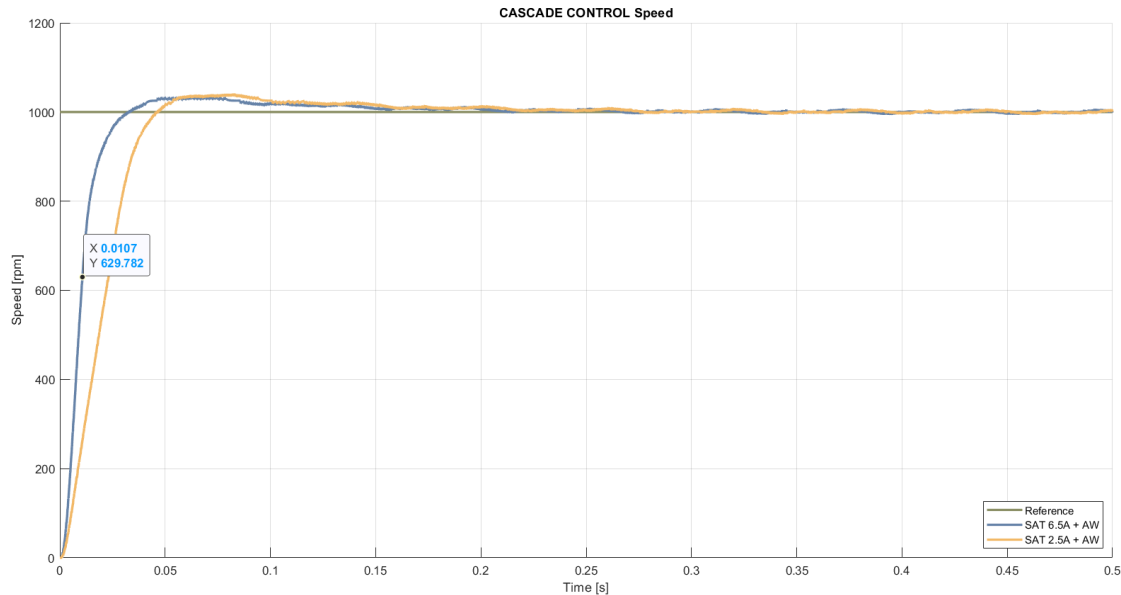


Figure 14. Comparison between speed step response with Saturation=2.5A and Saturation=6.5A.

The correspondent new current plot is shown in *Figure 15*.

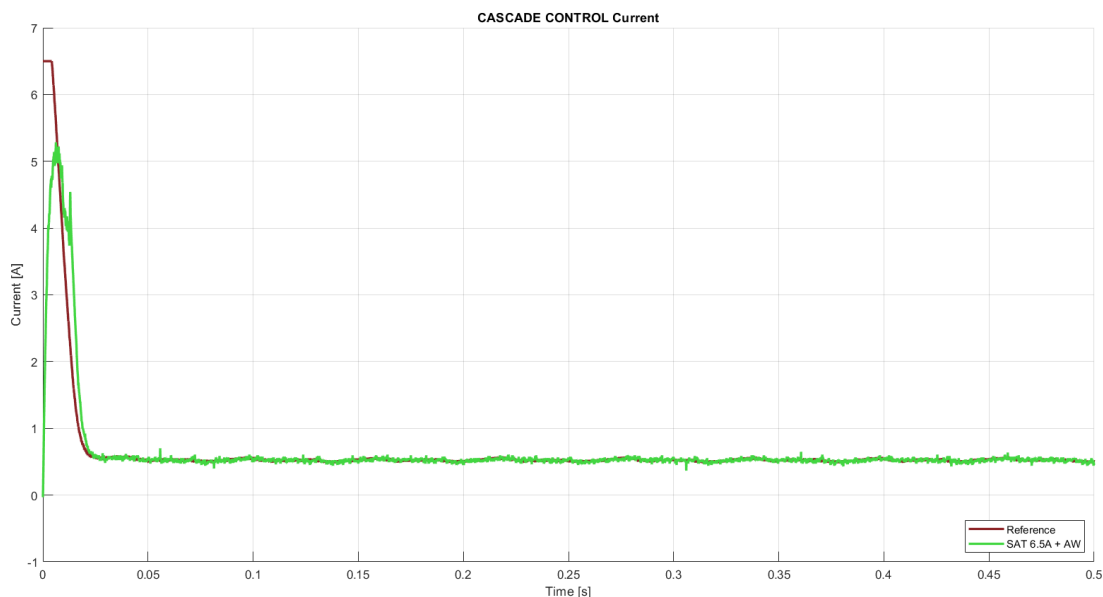


Figure 15. Current absorbed with 6.5A OCP limit.

It's very clear that, just by increasing a bit our motor capabilities, we can speed up our response and accomplish all dynamic requirements, at least in first approximation.

ESTIMATION OF SYSTEM'S PARAMETERS:

Motor parameters L_a , R_a , J and b are often provided by the constructors, but if we want to obtain better motor's performances, we need to estimate those data. Therefore, we have implemented the **Least Square Estimation (LSE) Algorithm**, which is one of the most used estimation procedures.

This algorithm finds the vector of motor parameters " α " that minimize the summation of the square of the errors, knowing the matrix of acquired data: " ϕ " and the set of output data " y ". We obtain the best estimation when $e = 0$ (*ideal case*), so we will get $\hat{\alpha} = (\phi^T \phi)^{-1} \phi^T y$, where $\hat{\alpha}$ is the vector of estimated parameters.

On MATLAB the algorithm was implemented through this function:

```
%% LMS FUNCTION
function [alpha,phi]= least_square(u,y,N)
for i=2:N
    phi(i,1)=y(i-1);
    phi(i,2)=u(i);
end
phiT = transpose(phi);
alpha = inv(phiT*phi)*phiT*y(1:N);
end
```

Figure 16. MATLAB function including the Least Square Estimation Algorithm.

Then we adapted the LSE algorithm for two different estimation cases: **La-Ra estimation** and **J-b estimation**.

La-Ra Estimation:

```
%% DATA PROCESSING
% input and output data
in=data(:,1);
out=data(:,2);

[alpha,phi]= least_square(in(1:N+1),out(1:N+1),N);

%alpha
Lstim = alpha(1,1)/alpha(2,1)*Tsample
Rstim = (1-alpha(1,1))/(alpha(2,1))

Lstim = cast(Lstim,"double");
```

Figure 17. Matlab code for Ra-La estimation.

In order to stimulate only the electrical dynamics, just making our motor vibrates so that no BEMF was generated, we wanted our system to track a high-frequency square reference wave, which mean value was null.

Thanks to those hypothesis and by discretizing only the electrical dynamics, using the backward difference method, we have studied the input-output relation and found the estimated L_a and R_a , which are:

$$\begin{cases} L_a = \frac{\alpha_1}{\alpha_2} T_s = \frac{0.9484}{0.0155} 0.0001 = 6.1 \cdot 10^{-3} H \\ R_a = \frac{1 - \alpha_1}{\alpha_2} = \frac{1 - 0.9484}{0.0155} = 3.3369 \Omega \end{cases} \Rightarrow \tau_{el} = \frac{L_a}{R_a} = 1.8 ms$$

We also tried to identify some differences between datasheet and estimated parameters in terms of tracking with respect to a current reference trajectory produced by correlated Simulink model, as we can see in the *Figure 18*.

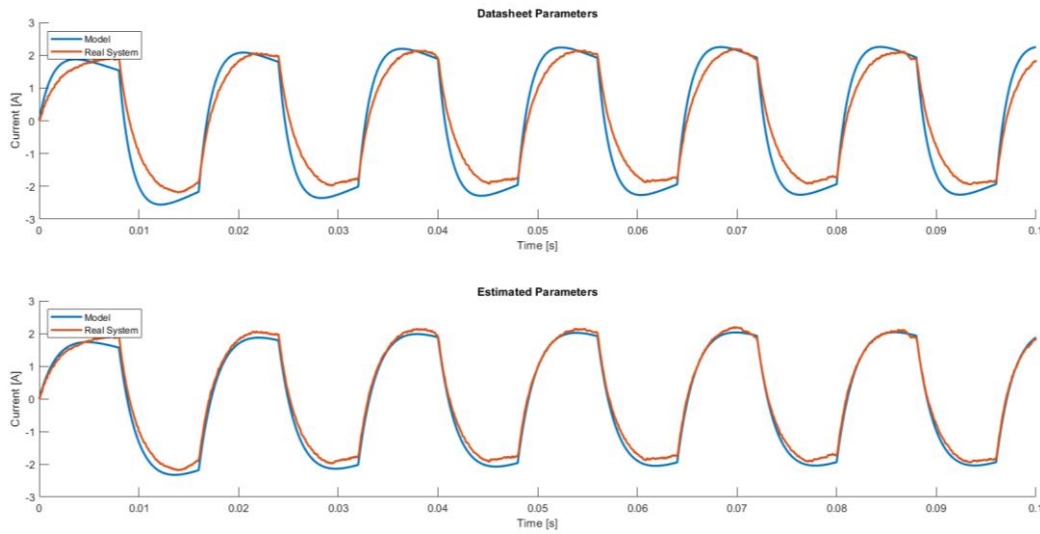


Figure 18. Comparison between datasheet parameters and estimated ones, La-Ra.

We can see that the estimated parameters help us in following better the reference trajectory derived from the model and this highlights the importance of estimating those constants and not relying blindly on datasheet's parameters.

J-b Estimation:

```
%% DATA PROCESSING
% input and output data
in=data(:,1);
out=data(:,2);

[alpha,phi]= least_square(in(1:N+1),out(1:N+1),N);
%alpha
Jstim=alpha(1,1)/alpha(2,1)*TSample
bstim=(1-alpha(1,1))/(alpha(2,1))

Jstim = cast(Jstim,"double");
bstim = cast(bstim,"double");
```

Figure 19. Matlab code for J-b estimation.

For J&b estimation, we took a lower-frequency square reference wave so that the mechanical dynamics could be stimulated and the electrical one could be neglected. Thanks to those hypothesis and by discretizing only the mechanical dynamics, using the backward difference method, we have studied the input-output relation, finding the estimated J and b, which are:

$$\begin{cases} J = \frac{\alpha_1}{\alpha_2} T_s = \frac{0.9990}{1.1955} 0.0001 = 8.3561 \cdot 10^{-5} \text{ kg} \cdot \text{m}^2 \\ b = \frac{1 - \alpha_1}{\alpha_2} = \frac{1 - 0.9990}{1.1955} = 8.5764 \cdot 10^{-4} \frac{\text{Nm}}{\text{rad/s}} \end{cases} \Rightarrow \tau_{mecc} = \frac{J}{b} = 97.4 \text{ ms} \gg \tau_{el}$$

Then, we tried again to identify some differences between datasheet and estimated parameters in terms of tracking with respect to a speed reference trajectory produced by correlated Simulink model, as we can see in the *Figure 20*.

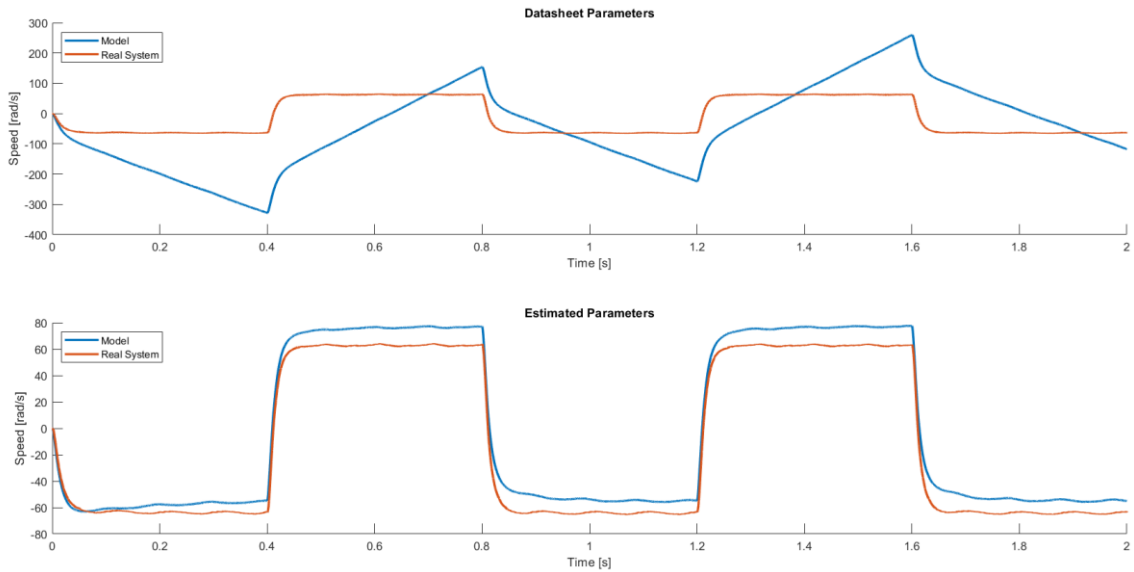


Figure 20. Comparison between datasheet parameters and estimated ones, J-b.

We can see again that the estimated parameters track better the reference trajectory derived from the model.

DEADBEAT CONTROL:

Subsequently we implemented the Deadbeat control scheme. The Deadbeat design imposes some desired properties to the output signal $Y(z)$, as long as the input reference $V(z)$ is a classical function (as a step, a ramp, a parabola, ...).

We implemented the simplified Deabeat design procedure, which is based on the following assumptions:

- only step references as input ($V(z) = \frac{1}{1-z^{-1}}$), indeed we considered always a step input
- plant dynamic $G_p(z)$ is stable and minimum phase (no zeros or poles outside the unit circle)
- desired dynamics $G_m(z) = z^{-k}$ with $k \geq r_p$, with r_p = relative degree of $G_p(z)$

The Deadbeat control scheme is shown in Figure 21.

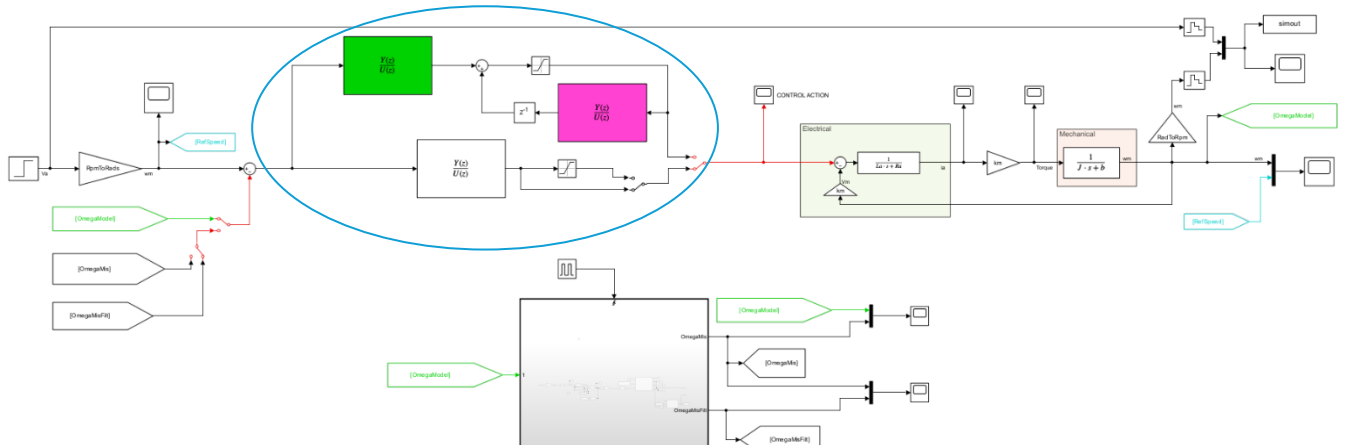


Figure 21. Deadbeat control scheme.

In our case the entire closed loop transfer function of the plant (which includes both the electrical and the mechanical dynamics) is described by the following equation:

$$G_p(s) = \frac{G_{el}(s)k_m G_{mecc}(s)}{1 + G_{el}(s)k_m G_{mecc}(s)} = \frac{k_m}{L_a J s^2 + (R_a J + L_a b)s + b R_a + k_m^2}$$

After having discretized $G_p(s)$, through MATLAB, and by choosing $T_s = 0.1 \text{ ms}$, we obtained the following $G_p(z)$:

$$G_p(z) = \frac{0.02773z + 0.02722}{z^2 - 1.945z + 0.9458} = \frac{0.027732(z + 0.9816)}{(z - 0.9931)(z - 0.9524)}$$

From the theory we know that the expression of a controller $D(z)$ in the discrete-time is:

$$D(z) = \frac{1}{G_p(z)} \cdot \frac{G_m(z)}{1 - G_m(z)}$$

By choosing $k = 1 \geq r_p$ and recalling that $G_m(z) = z^{-k}$, we obtained:

$$D(z) = \frac{1}{G_p(z)} \cdot \frac{z^{-1}}{1 - z^{-1}}$$

After substituting all the values, we finally obtained the expression of the Deadbeat controller:

$$D(z) = \frac{36.06z^2 - 70.15z + 34.1}{z^2 - 0.0184z - 0.9816} = \frac{36.059(z - 0.9931)(z - 0.9524)}{(z - 1)(z + 0.9816)}$$

After all these computations, we created a transfer function block in Simulink that represents our Deadbeat regulator $D(z)$, and then we **compared** the speed step response from the **ideal** case (obtained with the simulation model) with the one of the **real** case (obtained through our real motor), as shown in *Figure 22*. For this comparison we have exploited the Deadbeat controller **considering only the estimated parameters of the DC motor**.

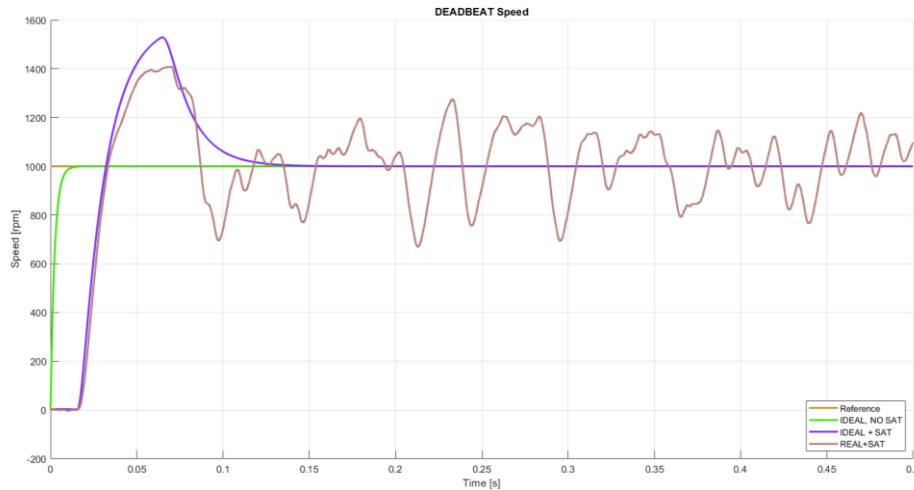


Figure 22. Ideal vs Real step response with Deadbeat controller without AW.

The ideal response without the saturation presents, as expected, a perfect behaviour because under this condition the motor can reach high voltages without any limit, bringing the response to the steady state very quickly. However the differences between the real response and the ideal response, both with the saturation limit, are **significant**: the real motor response presents overshoots and undershoots and seems that it will never reach the reference value provided as input. This is because the motor is too stressed, since the Deadbeat control scheme demands the **fastest step response possible**. This phenomenon is also justified by looking to the control action behaviour, shown in *Figure 23*.

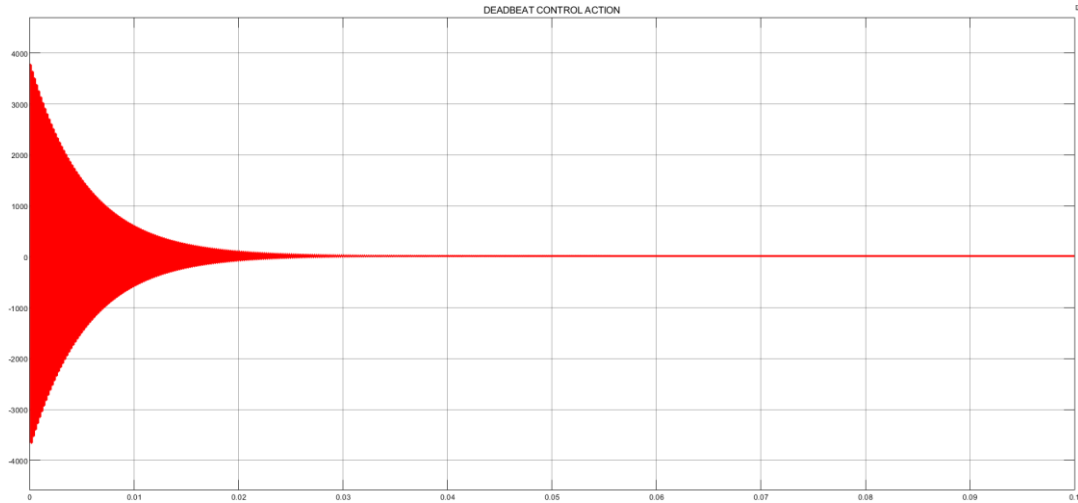


Figure 23. Plot of the control action with Deadbeat control without SAT without AW.

No common motor is able to reach those voltage values without ruining itself during operation. In fact, the biggest problem of the Deadbeat control scheme is that, by trying to bring the output to the steady state value in the smallest possible number of time steps, it will put under **unreachable efforts** the control input of the motor, implying very large control signals.

To reduce this control action issue, we have introduced a standard anti-windup scheme, considering now the real and the ideal responses with the saturation limit of 24V.

This scheme is very simple and allows us to rewrite the regulator $D(z)$ in order to have some benefits. The numerator and the denominator of the controller $D(z)$ are now called respectively $B(z)$ and $A(z)$. Then we can exploit a polynomial function $F(z)$ and, following the anti-windup scheme 2 from the theory, we inserted $\frac{B(z)}{F(z)}$ into the green transfer function block and $\frac{F(z)-A(z)}{F(z)}$ into the pink one, as shown in *Figure 24*. To implement correctly this anti-windup scheme, $F(z)$ must have the following properties:

1. be an asymptotically stable polynomial
2. $\deg\{F(z)\} \geq \deg\{B(z)\}$
3. $\frac{B(1)}{F(1)} \geq 0$

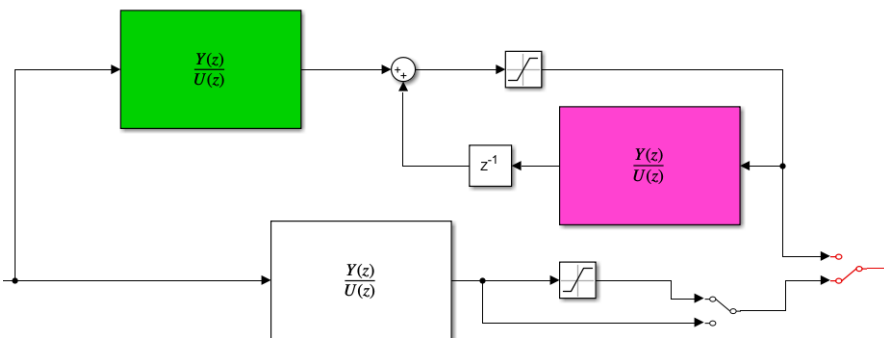


Figure 24. Antiwindup control scheme. Lower path = original regulator scheme, Upper path = new regulator scheme.

By just implementing this antiwindup technique and choosing: $F(z) = 20(z - 0.93)(z - 0.96)$ we can achieve the result shown in *Figure 25*, in terms of speed.

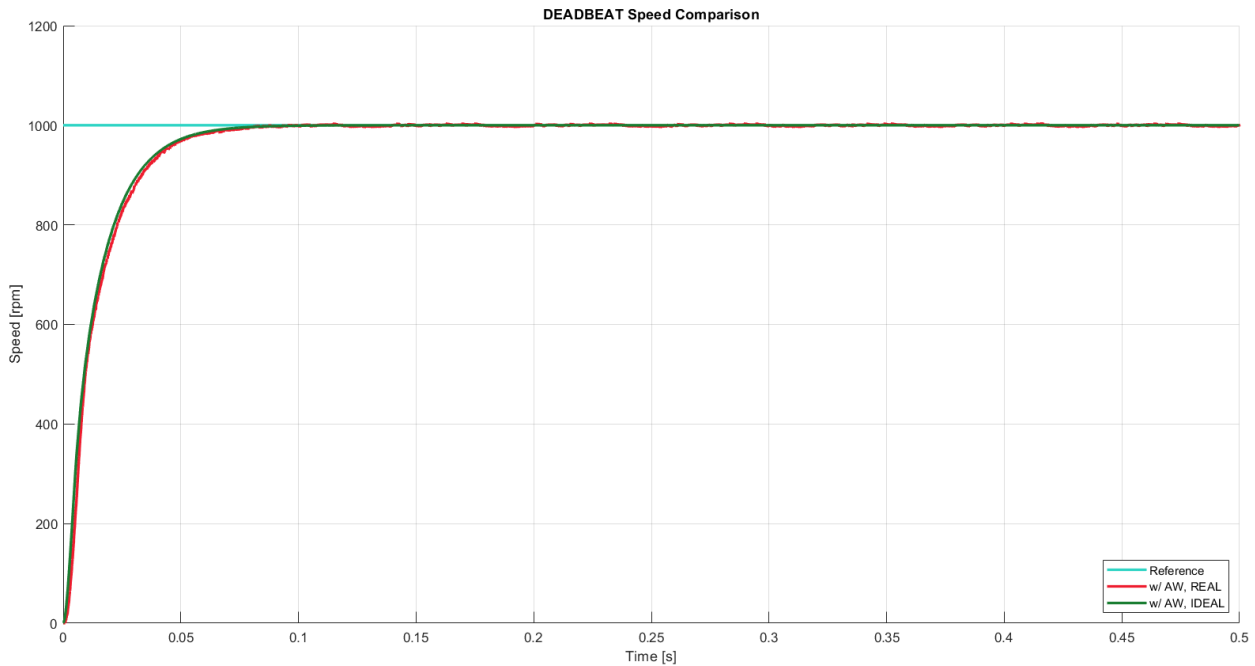


Figure 25. Ideal vs Real step response with Deadbeat controller with AW.

Now, as we can observe, the real speed step response it's almost the same as the ideal one. Performances of the system, in terms of signal stability, has been greatly improved by introducing the anti-windup system.

The correlated control action plot is shown in *Figure 26*:

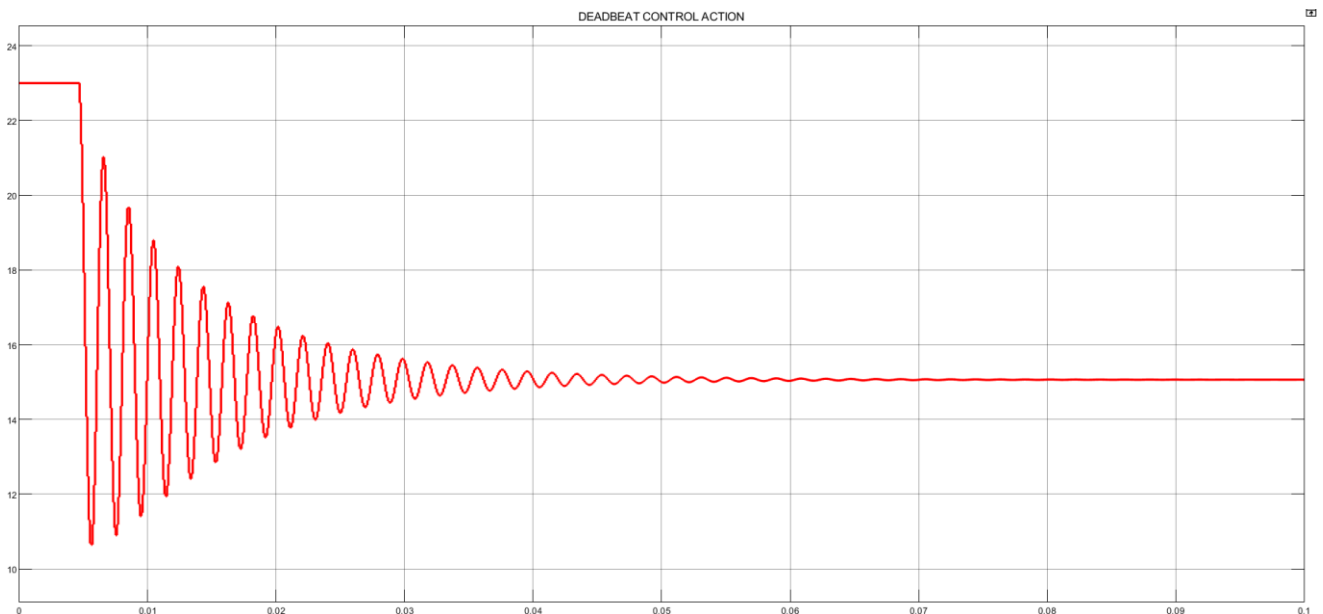


Figure 26. Plot of the control action with Deadbeat control with AW.

It's evident that the new control action behaviour it's way better (look to the scale of the graph) and therefore less effort is required to the motor.

DAHLIN CONTROL:

The last control scheme that we have implemented was based on the Dahlin control strategy, which is very similar to the Deadbeat control. The main difference between the two control methods is that Dahlin generates smoother exponential responses. Furthermore, the control action which governs the Dahlin controller is less aggressive with respect to the Deadbeat one.

The Dahlin control scheme is shown in *Figure 27*.

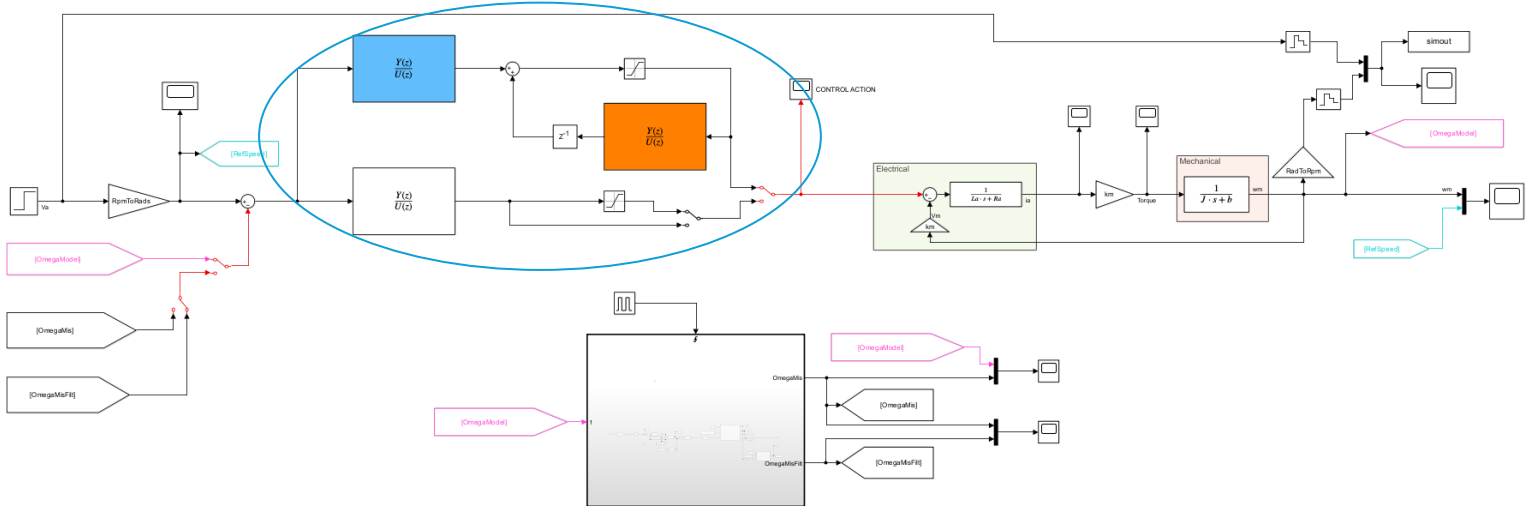


Figure 27. Dahlin Control Scheme.

In this case, the desired behavior of the closed loop system is not just equivalent to a time delay as in Deadbeat; it's instead the response $Y(s)$ of a first order system with a time delay θ and a time constant τ :

$$Y(s) = \frac{e^{-\theta s}}{\tau s + 1} \frac{1}{s}$$

Then, choosing $T_s = 0.1 \text{ ms}$, we have discretized $Y(s)$, obtaining:

$$Y(z) = \frac{(1 - e^{-T_s/\tau}) z^{-N-1}}{(1 - z^{-1})(1 - e^{-T_s/\tau} \cdot z^{-1})}; \quad \text{with } N = \text{floor}(\theta/T_s)$$

Now we considered a step input $V(z)$:

$$V(z) = \frac{1}{(1 - z^{-1})} \rightarrow \frac{Y(z)}{V(z)} = \frac{(1 - e^{-T_s/\tau}) z^{-N-1}}{(1 - e^{-T_s/\tau} \cdot z^{-1})} = G_m(z)$$

Just like the Deadbeat case, the expression of the controller $D(z)$ in the discrete-time is:

$$D(z) = \frac{1}{G_p(z)} \cdot \frac{G_m(z)}{1 - G_m(z)} \text{ and substituting we obtain: } D(z) = \frac{1}{G_p(z)} \cdot \frac{(1 - e^{-T_s/\tau}) z^{-N-1}}{1 - e^{-T_s/\tau} \cdot z^{-1} - (1 - e^{-T_s/\tau}) z^{-N-1}}$$

Reconnecting to our case, the entire discrete-time closed loop transfer function of the plant with the Dahlin control scheme becomes:

$$G_p(z) \approx \frac{0.001155(z + 0.9816)}{(z - 0.9931)(z - 0.9524)}$$

Then, we imposed $\theta = 0.1ms$ and $\tau = 20ms$, in order to obtain a delay of exactly one sampling time:

$$\theta = T_s = 0.1ms \rightarrow N = \text{floor}\left(\frac{\theta}{T_s}\right) = 1$$

Recalling that $G_m(z) = \frac{(1 - e^{-T_s/\tau})z^{-N-1}}{(1 - e^{-T_s/\tau \cdot z^{-1}})}$ and substituting all the values, we obtained:

$$G_m(z) \approx \frac{0.004988}{z(z - 0.995)}$$

Lastly, the expression of the Dahlin controller resulted to be:

$$D(z) = \frac{4.316z^2 - 8.397z + 4.082}{z^3 - 0.01342z^2 - 0.9817z - 0.004896} \approx \frac{4.3163(z - 0.9931)(z - 0.9524)}{(z - 1)(z + 0.9816)(z + 0.004988)}$$

Then, we compared the ideal speed step response (obtained with the simulation model) with the real speed step response (obtained through our physical motor), exploiting the Dahlin controller and **considering only the estimated parameters of the motor**, as shown in *Figure 28*.

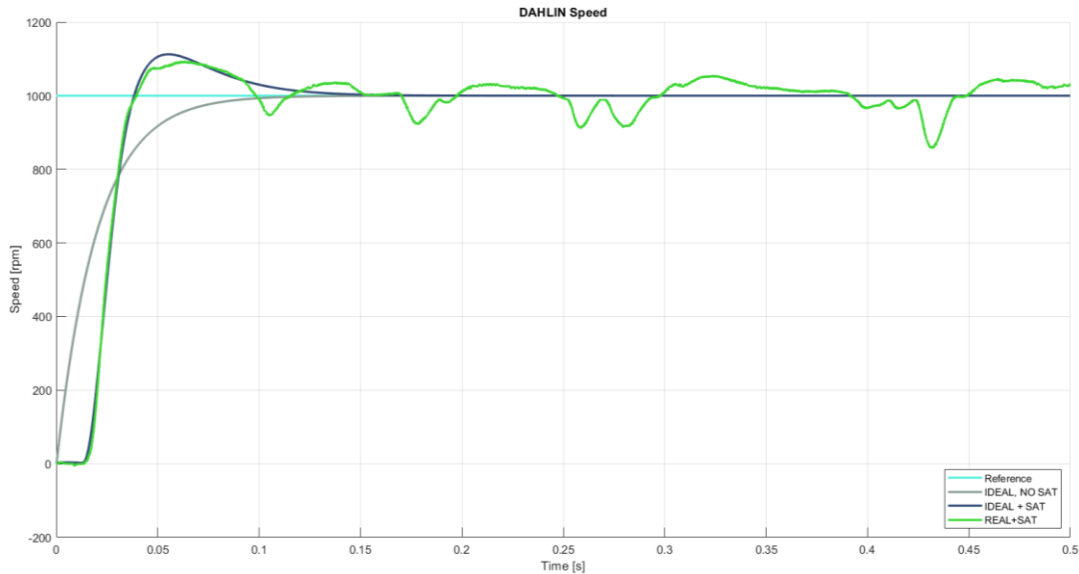


Figure 28. Ideal vs Real step response with Dahlin controller without AW.

As in the Deadbeat case, also with the Dahlin control, we can observe that the real speed response shows an undesired behaviour, indeed it does not follow correctly the reference input represented by the ideal response with the implementation of the saturation limit. This problem is caused again by the high effort required to the motor due to a high control action, as shown in *Figure 29*.

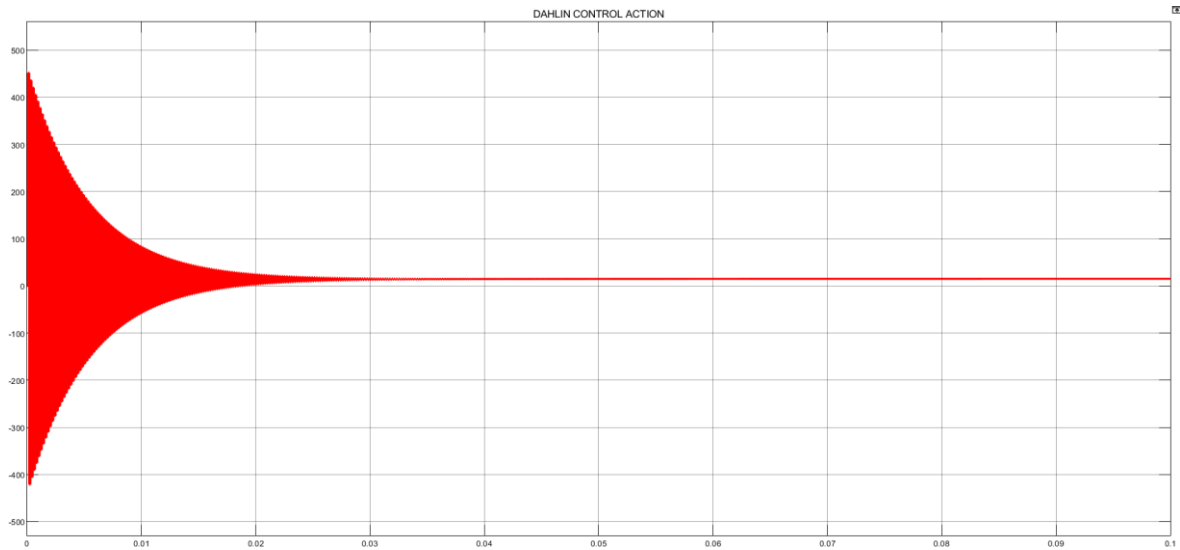


Figure 29. Plot of the control action correspondent to the ideal speed step response with Dahlin control, without SAT without AW.

As we can see, the control action of our system, which is essentially the effort required to our motor, is very demanding even if it's much lower (look to the scale of the graph) compared to the Deadbeat case.

In order to solve this problem, and **considering now the real and the ideal responses with the saturation limit of 24V**, we implemented an **anti-windup scheme** – the same type used before with Deadbeat – but this time we chose $F(z) = 25z(z - 0.97)(z - 0.95)$ as the polynomial function, obtaining the result in Figure 30.

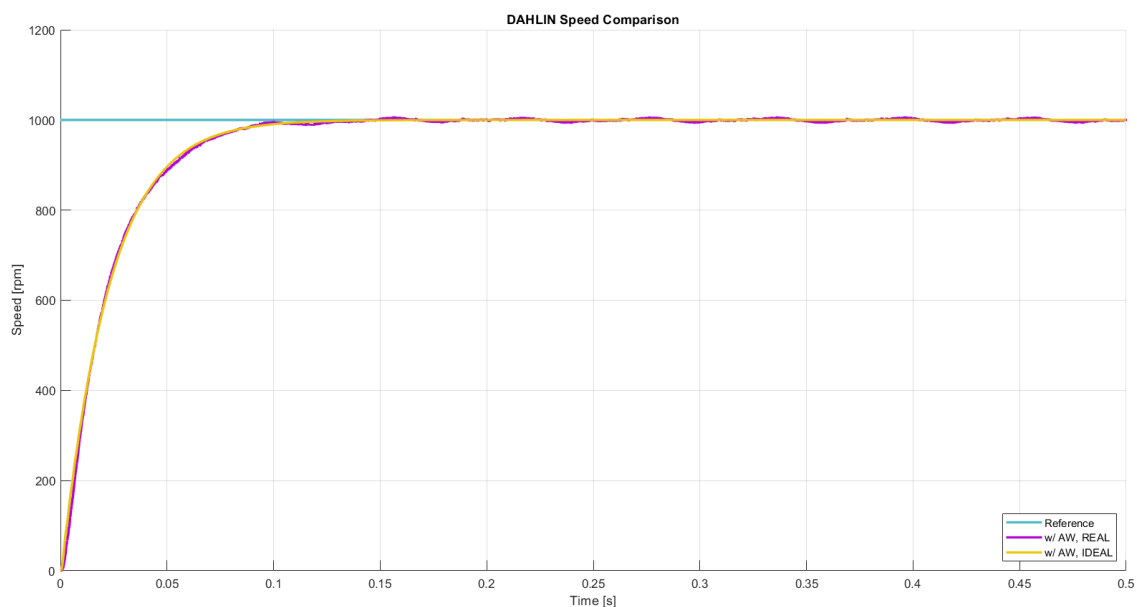


Figure 30. Ideal vs Real step response with Dahlin controller with Antiwindup

Now we can see how, thanks to the use of the anti-windup system, the overall performance has improved, in fact the signal does not present excessive overshoots, following almost perfectly the ideal behaviour, and it settles more quickly to the reference value.

The control action obtained with the anti-windup scheme is shown in *Figure 31*:

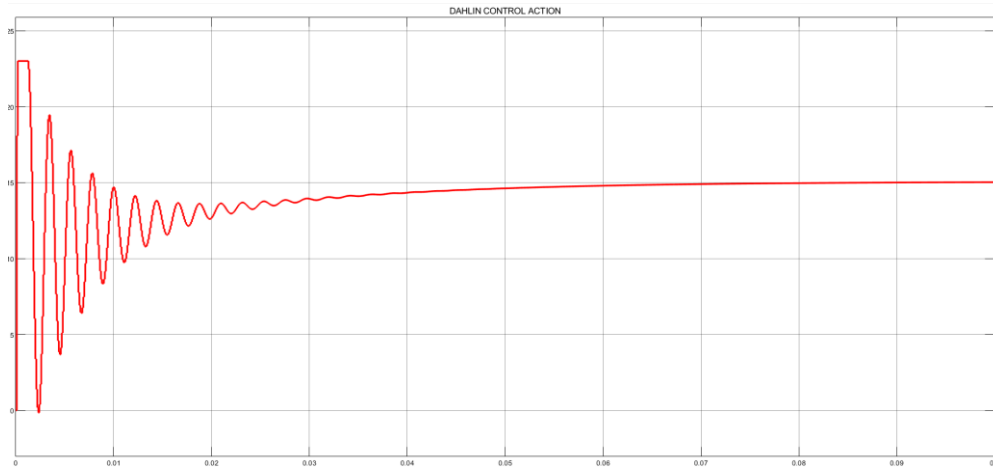


Figure 31. Plot of the control action with Dahlin control with AW.

We can observe that the situation is very good now. The initial oscillations are still there, but it's clear that the system is less stressed since the control action is limited within a very minimal range of values (look to the scale of the graph) with respect to the previous situation without the anti-windup.

CASCADE CONTROL WITH ESTIMATED PARAMETERS:

Subsequently, with the estimation of our motor's parameters carried out through the **Least Square Estimation Algorithm**, we jumped back to the first control scheme implemented (PI-cascade control scheme) and we checked for different behaviours. It's known, from the theory, that estimating the system parameters gives us much more accurate motor constants than those provided by manufacturers and consequently this will help us in creating more suitable controllers to adjust our loop; therefore the parameters' estimation of our DC motor will contribute in improving the performance of the system. To confirm what is known from theory, we compared motor step responses with datasheet parameters and with estimated ones, respectively in terms of speed and current. We first compared the speed response with the estimated parameters and with the datasheet parameters of the motor, as shown in *Figure 32*.

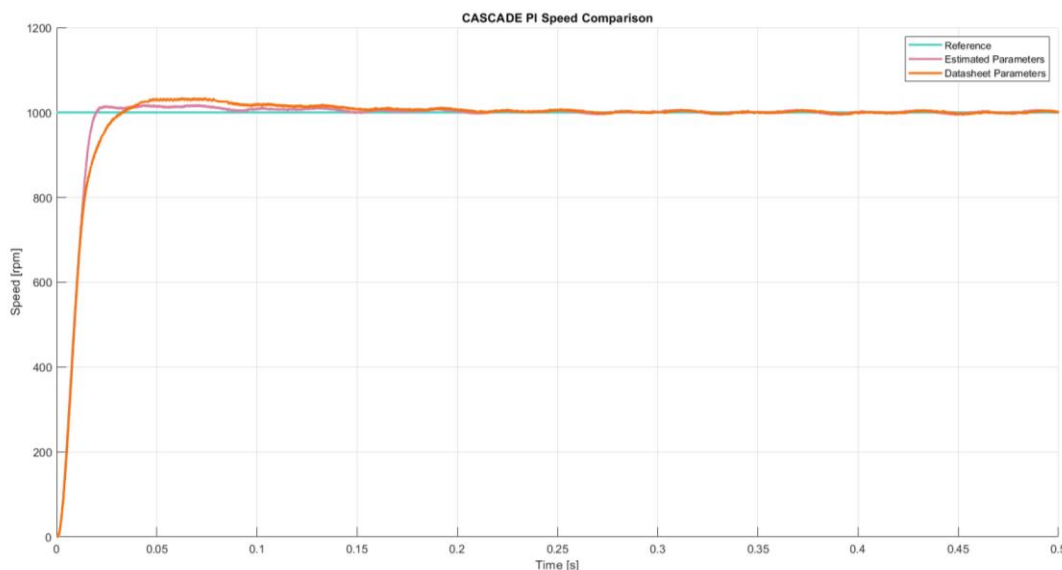


Figure 32. Estimated vs Datasheet parameters, Speed response with PI-cascade control.

Then we compared the current response with the estimated parameters and with datasheet parameters of the motor, as shown in *Figure 33*.

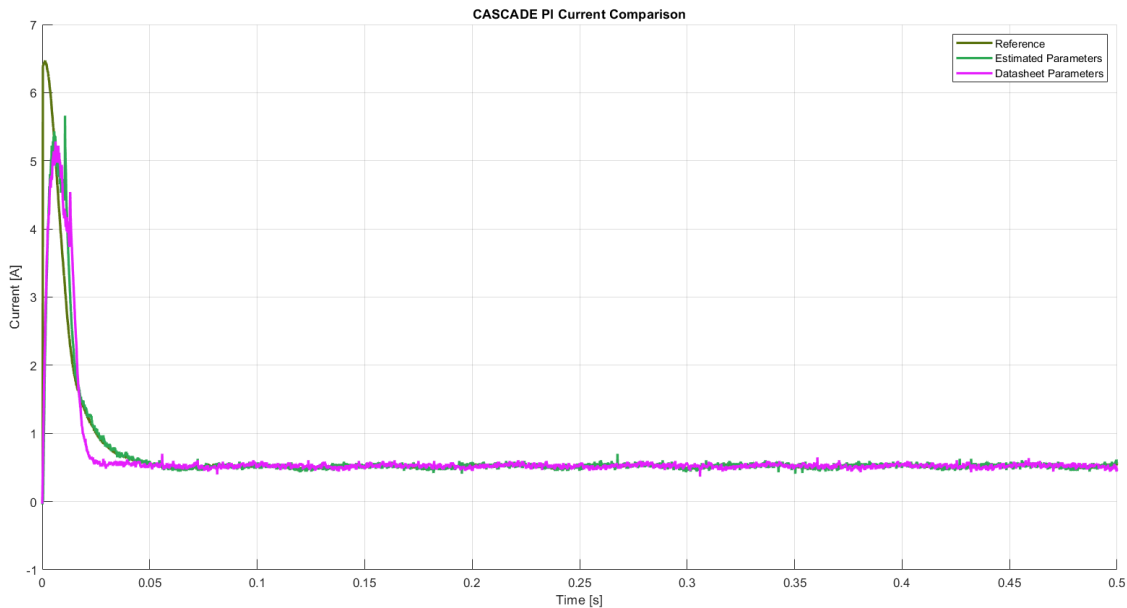


Figure 33. Estimated vs Datasheet parameters, Current response with PI-cascade control.

As expected, we can observe that tracking performances using estimated parameters of the motor are much better than those obtained by using datasheet parameters.

OTHER PLOTS & FEATURES:

RESISTIVE LOAD:

In this section of the report, we have inserted some other plots that help us to understand better the behavior of our system. First, we tested the reaction of our DC motor after the introduction of a braking resistive load during its operation. The resistive load generates a resistive torque by the current circulation in the windings after its connection to the motor. The results in terms of speed and current are shown in *Figure 34* and *Figure 35* respectively.

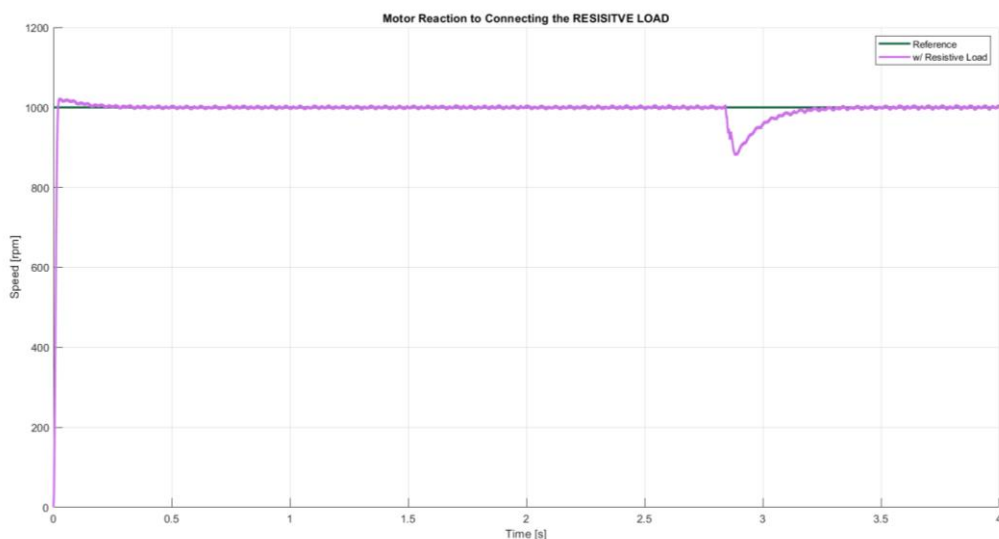


Figure 34. Motor speed during the introduction of the load.

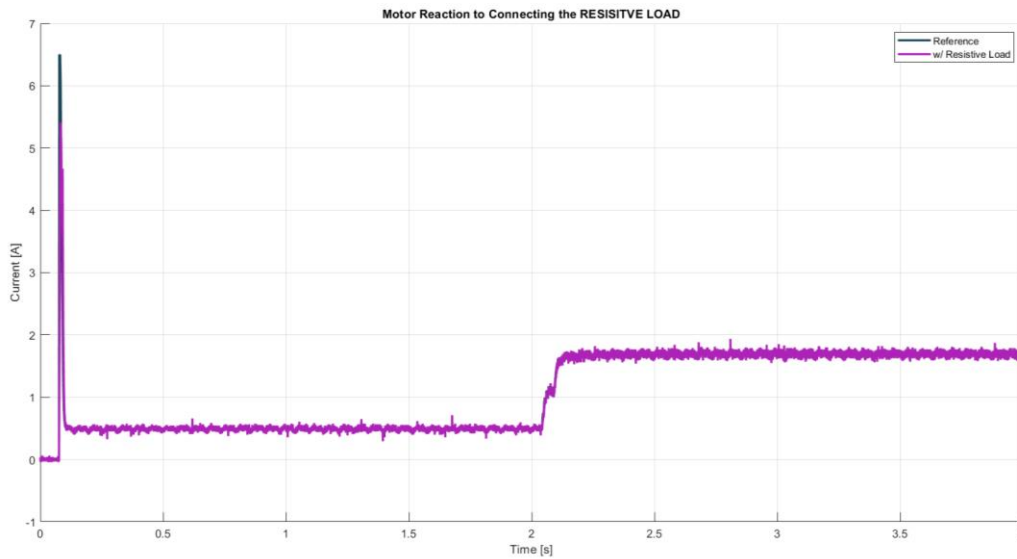


Figure 35. Motor current during the introduction of the load.

We can observe that at the starting the current compensates only the back emf and the friction and then, as soon as we attach the load, the current increases in order to compensate also the counter torque. This test demonstrates that, even with the presence of a disturbance represented by the introduction of the resistive load, the system is stable, reaching in any case the target reference. Although the two events shown in the above plots do not appear to be synchronized, in reality it is not like this. The reason behind this lack of synchronism is that the two data were taken in two different acquisition times.

STAIR REFERENCE:

Until now, all measurements have been collected providing a step reference speed of 1000 rpm as the input reference signal. Then, we decided to generate a new plot with the aim of highlighting the benefits introduced by the anti-windup scheme. For this test we considered our DC motor with the estimated parameters and its associated PI-Cascade control scheme, which has been already designed previously. Then we defined a **speed reference trajectory composed of several steps of increasing value over time** and we observed how the system reacted, as shown in Figure 36.

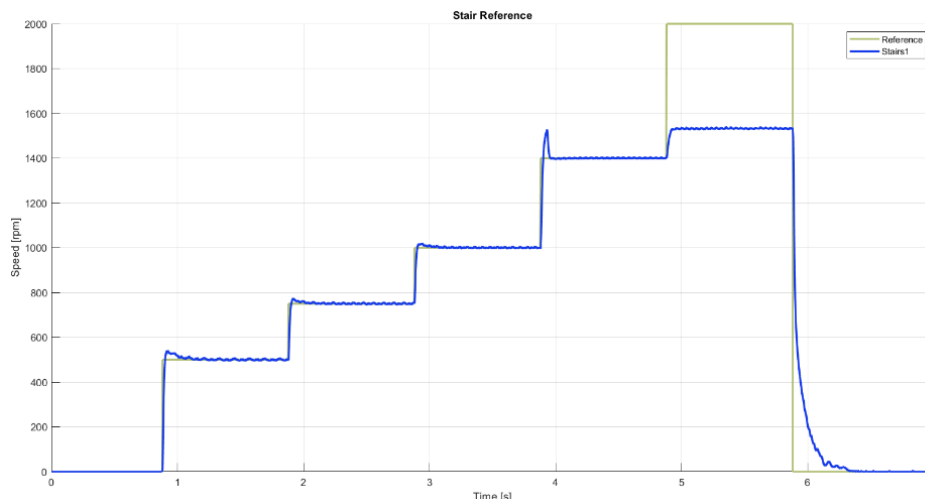


Figure 36. Speed with "stair" reference signal.

From the figure we can see that, in the last speed stair, the motor is **not able** to reach 2000 rpm because the imposed saturation limit leads the control action to saturate, preventing our DC motor from reaching that high speed. Therefore, in this condition, the integrator should “charge itself” and, after this, when the stair switch to **0 rpm target speed**, the output of the system should wait for the integrator to discharge, trying then to reach the reference. Instead, it’s clear from the plot that the **desaturation problem** of the control action, which would take time (disadvantage), is **avoided** thanks to the introduction of the antiwindup scheme in our regulator and the system reacts instantaneously, following the speed reference trajectory as soon as it becomes trackable again (returning under 1500 rpm).

FEED-FORWARD

Feedforward control represent an essential control strategy used to improve tracking capabilities of the system when faster trajectories are specified. Our project also includes two Feedforward actions, circled in *Figure 37*.

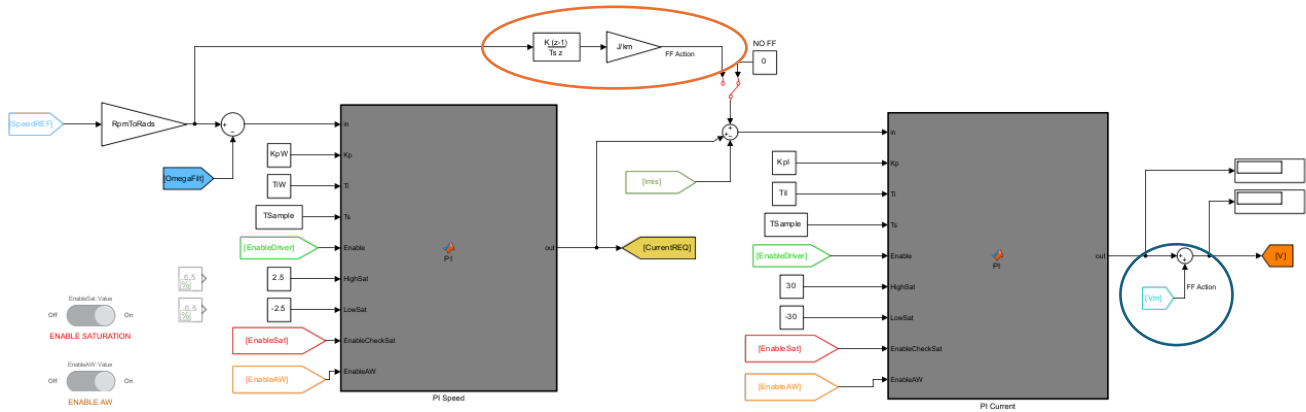


Figure 37. PI-Cascade control scheme with Feed-Forward actions.

This process consists generally in implementing additional functions by inverting our original dynamics $G(s)$ ($C_a(s) = \frac{1}{G(s)} \Rightarrow Y(s) = Y_d(s)$), but for this reason we must satisfy certain conditions (for example, we must be sure that our $G(s)$ has null relative degree so that $G^{-1}(s)$ can be computed). However, those actions could also be calculated on the basis of the known desired acceleration or speed without actually inverting our plant transfer function.

In the PI control scheme we have the possibility to inject these two feedforward actions:

- $v_M(t) = k_m \omega(t)$
- $i_{ff}(t) = \frac{J}{k_m} \dot{\omega}_d(t)$